

Number System

- Number is a Math value used for counting or measuring or labelling of.
- Number System is defined as a system of writing to express numbers. It is one the mathematical notation for representing no. of a given set by using digits.

Following are some N.S:- i) Decimal N.S.

ii) Binary N.S.

iii) Octal N.S.

iv) Hexadecimal N.S.

I) Decimal Number System- No. used in day-to-day life.

B

Base = 10 $\Rightarrow$ (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

$$\begin{array}{rcll} \text{eg- } 2586 & = & (2 \times 10^3) + (5 \times 10^2) + (8 \times 10^1) + (6 \times 10^0) \\ \begin{array}{c} \downarrow \quad \downarrow \\ 10^3 \quad 10^1 \\ \downarrow \quad \downarrow \\ 10^2 \quad 10^0 \end{array} & = & 2000 + 500 + 80 + 6 \\ & = & 2586 \end{array}$$

II) Binary N.S.- widely used in computers. All inputs given to a comp. are 1st converted to BNS b/c it understand only B.N.S..

Base = 2 $\Rightarrow$ (0, 1)

$$\begin{array}{rcll} \text{eg- } 10101 & = & (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ \begin{array}{c} \downarrow \quad \downarrow \quad \downarrow \\ 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0 \end{array} & = & 16 + 0 + 4 + 0 + 1 \\ & = & (21)_{10} = (10101)_2 \end{array}$$

$$\begin{aligned} (10100011)_2 &= 128 + 0 + 32 + 0 + 0 + 0 + 2 + 1 \\ &= (163)_{10} \end{aligned}$$

III) Octal N.S.:- Base $\Rightarrow 8$ (0, 1, 2, 3, 4, 5, 6, 7)

eg:- $(2057)_8 = 1024 + 0 + 40 + 7$
 $\begin{matrix} 1 & 1 & 1 & 1 \\ 8^3 & 8^2 & 8^1 & 8^0 \end{matrix} = (1071)_{10}$
 $\begin{matrix} 512 & 64 & 8 & 1 \end{matrix}$

eg:- $(2671)_8 = 1024 + 384 + 56 + 1$
 $= (1465)_{10}$

eg:- $(276)_8 = 128 + 64 + 6$
 $\begin{matrix} \uparrow \\ \text{invalid} \end{matrix} = (198)_{10} \times$

IV) Hexadecimal N.S.:- Base $\Rightarrow 16$ (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

16 Symbols
 or Character (Single)

10 - A	13 - D
11 - B	14 - E
12 - C	15 - F

$(1AF)_{16} = 256 + 160 + 15$ $= (431)_{10}$	$(7CF)_{16} = 1792 + 192 + 15$ $= (1999)_{10}$
---	---

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}
1	2	4	8	16	32	64	128	256	512	1024

2^{11}	2^{12}	8^0	8^1	8^2	8^3	8^4
2048	4096	1	8	64	512	4096

16^0	16^1	16^2	16^3	16^4
1	16	256	4096	65536

Converting from Decimal to Binary:-

$$\begin{array}{rcl}
 (25)_{10} & \Rightarrow & \begin{array}{r} 2 \ 25 \ 1 \\ 2 \ 12 \ 0 \\ 2 \ 6 \ 0 \\ 2 \ 3 \ 1 \\ 1 \end{array} \Rightarrow (11001)_2 \Rightarrow 16 + 8 + 0 + 0 + 1 \\
 & & = (25)_{10}
 \end{array}$$

$$\begin{array}{rcl}
 (42)_{10} & \Rightarrow & \begin{array}{r} 2 \ 42 \ 0 \\ 2 \ 21 \ 1 \\ 2 \ 10 \ 0 \\ 2 \ 5 \ 1 \\ 2 \ 2 \ 0 \\ 1 \end{array} \Rightarrow (101010)_2
 \end{array}$$

$$\begin{array}{rcl}
 (156)_{10} & \Rightarrow & \begin{array}{r} 2 \ 156 \ 0 \\ 2 \ 78 \ 0 \\ 2 \ 39 \ 1 \\ 2 \ 19 \ 1 \\ 2 \ 9 \ 1 \\ 2 \ 4 \ 0 \\ 2 \ 2 \ 0 \\ 1 \end{array} \Rightarrow (10011100)_2
 \end{array}$$

Decimal to Octal:-

$$\begin{array}{rcl}
 (952)_{10} & = & \begin{array}{r} 8 \ 952 \ 0 \\ 8 \ 119 \ 7 \\ 8 \ 14 \ 6 \\ 1 \end{array} \Rightarrow (1670)_8
 \end{array}$$

$$\begin{array}{rcl}
 (210)_{10} & = & \begin{array}{r} 8 \ 210 \ 2 \\ 8 \ 26 \ 2 \\ 3 \end{array} \Rightarrow (322)_8
 \end{array}$$

Converting Decimal to Hexadecimal:-

$$(428)_{10} = \begin{array}{r} 16 \ 428 \ 12 \\ 16 \ 26 \ 10 \\ 1 \end{array} \Rightarrow (1AC)_{16}$$

$$(188)_{10} = \begin{array}{r} 16 \ 188 \ 12 \\ 11 \end{array} \Rightarrow (BC)_{16}$$

Converting from a base other than 10 to base other than 10.

- 1) Convert the original no. to decimal no.
- 2) Convert the decimal no. to req. no. (with new base).

$$\text{eg: } (101110)_2 = ()_8 \Rightarrow 32 + 8 + 4 + 2 \\ \Rightarrow (46)_{10}$$

$$\begin{array}{r} 8 \ 46 \ 6 \\ 5 \end{array} \Rightarrow (56)_8$$

$$(1AB)_{16} = 256 + 160 + 11 \\ = (427)_{10}$$

$$(11010011)_2 = 128 + 64 + 16 + 2 + 1 \\ = (211)_{10}$$

$$\begin{array}{r} 2 \ 427 \ 1 \\ 2 \ 213 \ 1 \\ 2 \ 106 \ 0 \\ 2 \ 53 \ 1 \\ 2 \ 26 \ 0 \\ 2 \ 13 \ 1 \\ 2 \ 6 \ 0 \\ 2 \ 3 \ 1 \\ 1 \end{array} \Rightarrow (110101011)_2$$

$$16 \ 211 \ 3 \Rightarrow (D3)_{16}$$



Shortcut from Binary to Octal

- 1) Divide Binary digit into group of 3 digits (Starting from right)
- 2) Convert each group of 3 Binary Digit into one octal digit.

Decimal	HexaDecimal	Binary	Octal
0	0	0000	0
1	1	0001	1
2	2	0010	2
3	3	0011	3
4	4	0100	4
5	5	0101	5
6	6	0110	6
7	7	0111	7
8	8	1000	
9	9	1001	
10	A	1010	
11	B	1011	
12	C	1100	
13	D	1101	
14	E	1110	
15	F	1111	

$$\text{eg- } \underbrace{(101\ 110)}_{5\ 6}_2 \rightarrow (56)_8 \Rightarrow (56)_8$$

$$\underbrace{(1101\ 010)}_{15\ 2}_2 \rightarrow (152)_8$$

Short cut for Octal to Binary:-

- 1) Convert each octal digit to 3 digit Binary Numbers.
- 2) Combine all resulting Binary groups into Single B. Number

$$(562)_8 = (101\ 110\ 010)_2 \quad ; \quad 6751 = 110\ 111\ 101\ 001$$

Short for Binary to Hexadecimal:-

- 1) Convert each H.D. to 4 digit B. Number.
- 2) Combine all grps together

$$\begin{array}{c} \text{11101111} \\ \text{13 15} \\ \hline \end{array} = (1DF)_{16}$$

$$\begin{array}{c} 111\ 0010\ 1011 \\ \text{7 2 11} \\ \hline \end{array} = (72B)_{16}$$

Short for Hexadecimal to Binary:-

$$(2AB)_{16} = (0010\ 1010\ 1011)_2$$

$$(ABC)_{16} = (1010\ 1011\ 1110)_2$$

Fractional no. Conversion:- [① First Fraction to Decimal]

$$\begin{array}{c} (110.101)_2 \\ \text{421} \\ \hline 6 \end{array} \begin{array}{c} -1/2 -1/8 \\ \text{101} \\ \hline 5+0+1/8 \end{array} \Rightarrow 6 + \frac{5}{8} \Rightarrow 6 + 0.625$$

$$\Rightarrow \boxed{6.625}_{10}$$

$$(127.54)_8 = 64 + 16 + 7 + \frac{5}{8} + \frac{4}{64} \Rightarrow \boxed{127.6875}_{10}$$



$$2B.C4 \rightarrow 32 + 11 + \frac{1^2}{16} + \frac{4}{256} \rightarrow (43.765625)_{10}$$

$$(0.6875)_{10} = (?)_2$$

Base

	Fraction	integer
$0.6875 \times 2 =$	0.3750	1
$0.3750 \times 2 =$	0.7500	0
$0.7500 \times 2 =$	0.5000	1
$0.5000 \times 2 =$	0.0000	1

$$= (0.1011)_2$$

Binary Arithmetic:-

Addⁿ - $0 + 0 = 0$

$0 + 1 = 1$

$1 + 0 = 1$

$1 + 1 = 0 \text{ (and 1 carry to next higher column.)}$

$$\begin{array}{r} 10011 \\ 01001 \\ \hline 11100 \end{array}$$

$$\begin{array}{r} 10001 \\ 11101 \\ \hline 101110 \end{array}$$

$$\begin{array}{r} 101101 \\ 11001 \\ \hline 1000110 \end{array}$$

Subⁿ - $0 - 0 = 0$

$1 - 0 = 1$

$1 - 1 = 0$

$0 - 1 = 1 \text{ (Borrow from next column.)}$

$$\begin{array}{r} 1011011 \\ 10010 \\ \hline 1001001 \end{array}$$

$$\begin{array}{r} 101101 \\ 100111 \\ \hline 000110 \end{array}$$

$$\begin{array}{r} 1000101 \\ 101100 \\ \hline 0001001 \end{array}$$

mult $0 * 0 = 0$ 1101 1010
 $0 * 1 = 0$ 100 101
 $1 * 0 = 0$ 0000 1010
 $1 * 1 = 1$ $0000x$ $0000x$
 $1101xx$ $1010xx$
 110100 110010

Divi $0 / 0 = \text{undefined}$
 $0 / 1 = 0$
 $1 / 0 = \text{undefined}$
 $1 / 1 = 1$

BCD $\rightarrow$ Binary Coded Decimal $\rightarrow$ It is a method of representing Decimal no. in Binary, where each Decimal Digit is stored Separately using 4 bit Binary code. Each D. Digit is converted into its corresponding 4 bit B-code. Every digit is encoded separately instead of converting the entire D. no. into Binary.

D. No BCD

0 0000

1 0001

2 0010

3 0011

4 0100

5 0101

6 0110

7 0111

8 1000

9 1001

eg- $57 \Rightarrow (01010111)_{BCD}$
 0101 0111 BCD

1 Byte = 8 bit
 1 nibble = 4 bit = $\frac{1}{2}$ Byte

→ Packed BCD: It stores 2 Digit in a single 8-bit, each 4 bit (nibble) represents 1 D. digit.

eg:- $9 \rightarrow 1001$
 $3 \rightarrow 0011$
 $\rightarrow (93)_{10} \rightarrow (10010011)_{BCD}$

→ Unpacked BCD: It stores 1 D. digit per byte.

eg:- $9 \rightarrow 00001001$
 $3 \rightarrow 00000011$
 $\rightarrow (93)_{10} \rightarrow (0000100100000011)_{BCD}$

→ BCD-Addition: To add 2 BCD no., add corresponding 4-bit groups as normal binary numbers. If the sum of any 4-bit group is > than 1001 (9); then 0110 (6) is added as a correction to obtain a valid BCD.

eg:-	0010	23	0010	0011
	0101	<u>45</u>	0100	0101
	0111	68	0110	1000
			(6)	(8)

45	0100	0101
<u>37</u>	0011	0111
	0111	1100
	0110	

82 → 1000 0010

→ BCD - Subtraction: To subtract 2 BCD nos,
 Subtract the corresponding BCD Digits, if
 the result is not a valid BCD Digit,
 a correction factor $(0110)_2 (6)_{10}$ is applied.

$$\begin{array}{r}
 47 \quad 0100 \quad 0111 \\
 \underline{23 \quad 0010 \quad 0011} \\
 24 \quad \underline{\underline{00 \quad 0 \quad 0100}}
 \end{array}$$

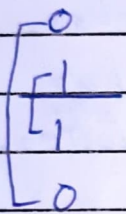
→ XS-3 Code: It is a non-weighted code,
 used to express decimal nos.

This code is derived from BCD code by adding
 $(0011)_2$ or $(3)_{10}$ to each BCD code.

0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100



→ Gray Code: It is a non weighted code, and its not arithmetic code that means there are no specific weight assigned to the bit position, it has a very special feature that has only one bit change, each time the decimal no. is increment as only 1 bit change at a time, It is also called as unit distance code. Used in K-map.



0	0	0
0	0	1
0	1	1
0	1	1
1	1	1
1	1	1
1	0	1
1	0	0

→ ASCII Code: Bits can also be coded to represent Alphabets, Numbers & Sp. character.
 → American Std. Code for Info. Interchange.
 → $2^7 = 128$ Char.

→ "A" is → 65 ; "Z" → 90

"a" is → 97 ; "z" → 122

"0" is → 48 ; "9" → 57

→ EBCDIC Code

- ↳ Extended Binary Coded Decimal Interchange Code.
- An 8-bit char. encoding system developed by IBM in 1963-64, for their mainframe & mid-range comp. systems.
- It translates human readable char. like letters, no. & sp. symbols into B.Code, so that large comp. processors can understand and store them.
- It uses 1 byte to represent a single char.
- It can represent upto 256 Unicode char. (Including Upper Case & (28) Lower case letters, nos, & sp. char.)
- Each 8-bit code is divided into 2 equal parts:-
 - 1) 1st 4 bits act as "zone",
 - 2) Last 4 bits act as "Digit".

Boolean Algebra:- It deals with the B.no. System, the variable used in the Boolean eqⁿ may assume only 2 possible value (0 & 1).

→ Logical Addition:- Symbol "+" is used for L.A. operator.

→ It is called "OR" operator.

→ Truth table for the Logical OR operator is:-

Input		Output
$A + B$	=	C
$0 + 0$	=	0
$0 + 1$	=	1
$1 + 0$	=	1
$1 + 1$	=	1

→ Logical Multiplier:- Symbol "." used for multiplication operator.

→ Also called "AND" operator.

→ Truth table for the Logical "AND" operator is:-

Input		Output
$A . B$	=	C
$0 . 0$	=	0
$0 . 1$	=	0
$1 . 0$	=	0
$1 . 1$	=	1

⇒ Complementation:- It is a unary operation which is defined on a single variable.

→ Symbol "=" is

→ It is normally used for Complementation operators.

→ Also known as "NOT" operator

→ Thus we write $\bar{A}$, meaning take complement of A.

Input	Output
A	$\bar{A}$
0	1
1	0

⇒ Operator Precedence:-

1) Evaluation $L \rightarrow R$

2) Parenthesis "()" are first evaluated.

3) All complement operation are performed next
(NOT)

4) All [AND (.)]

5) All [OR (+)].

⇒ Postulates of Boolean Algebra:-

1) a) $A = 0$; if and only - if A is not equal to 1.

b) $A = 1$; " " " " A $\neq 0$.

2) a) $x + 0 = x$

b) $x \cdot 1 = x$

3) Commutative Law:- a) $x + y = y + x$

b) $x \cdot y = y \cdot x$



4) Associative Law:- a) $x + (y + z) = (x + y) + z$
 b) $x \cdot (y \cdot z) = (x \cdot y) \cdot z$

5) Distributive Law:- a) $x \cdot (y + z) = x \cdot y + x \cdot z$
 b) $x + (y \cdot z) = (x + y) \cdot (x + z)$

6) $x + \bar{x} = 1$
 $x \cdot \bar{x} = 0$

⇒ The Principle of Duality:-

In Boolean Algebra, there is a precise duality b/w operators $(\cdot)$ AND, $(+)$ OR and digits "0" and "1". The implication of this duality is that any theorem in Boolean Algebra has dual obtainable by interchanging $(+)$ with $(\cdot)$ & "0" with "1", thus if a particular theorem is proved, its dual theorem automatically holds and need not to be proved separately.

1) Theorem-11:- $x + x = x$

LHS $\rightarrow x + x$

$(x + x) \cdot 1$ $[x \cdot 1 = x]$

$(x + x) \cdot (x + \bar{x})$ $[x + \bar{x} = 1]$

$x + x \cdot \bar{x}$ $[x + y \cdot z = (x + y) \cdot (x + z)]$

$x + 0$ $[x \cdot \bar{x} = 0]$

RHS $\rightarrow x$



II Theorem-21- $x + 1 = 1$

$$\text{LHS} \rightarrow x + 1$$

$$(x + 1) \cdot 1 \quad [x \cdot 1 = x]$$

$$(x + 1) \cdot (x + \bar{x}) \quad [x + \bar{x} = 1]$$

$$x + 1 \cdot \bar{x} \quad [x + y \cdot z = (x + y) \cdot (x + z)]$$

$$x + \bar{x} \cdot 1 \quad [x \cdot y = y \cdot x]$$

$$x + \bar{x} \quad [x \cdot 1 = x]$$

$$\text{RHS} \rightarrow \boxed{1} \quad [x + \bar{x} = 1]$$

III Theorem-31- $x + x \cdot y = x$

$$\text{LHS} \rightarrow x + x \cdot y$$

$$x \cdot 1 + x \cdot y \quad [x \cdot 1 = x]$$

$$x \cdot (1 + y) \quad [x \cdot (y + z) = x \cdot y + x \cdot z]$$

$$x \cdot (y + 1) \quad [x + y = y + x]$$

$$x \cdot 1 \quad [x + 1 = 1]$$

$$\text{RHS} \rightarrow \boxed{x} \quad [x \cdot 1 = x]$$

→ The theorem of Boolean Algebra can also be proved by the Truth Table

$$\Rightarrow x + x \cdot y = x$$

equal. (=)

x	y	$x \cdot y$	$x + x \cdot y$
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1



$$\Rightarrow x \cdot (x + y) = x$$

equal(=)			
x	y	$x + y$	$x \cdot (x + y)$
0	0	0	0
0	1	1	0
1	0	1	1
1	1	1	1

$$\Rightarrow \bar{x} = x$$

equal(=)		
x	$\bar{x}$	$\bar{\bar{x}}$
0	1	0
1	0	1

$$\Rightarrow x \cdot (\bar{x} + y) = x \cdot y$$

equal(=)					
x	y	$\bar{x}$	$\bar{x} + y$	$x \cdot (\bar{x} + y)$	$x \cdot y$
0	0	1	1	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	1	0	1	1	1

$$\Rightarrow x + \bar{x} \cdot y = x + y$$

equal(=)					
x	y	$\bar{x}$	$\bar{x} \cdot y$	$x + \bar{x} \cdot y$	$x + y$
0	0	1	0	0	0
0	1	1	1	1	1
1	0	0	0	1	1
1	1	0	0	1	1

De-Morgan's Law:-

→ De Morgan (a mathematician) developed a pair of important rules regarding group complementation (complement of a group of terms) in Boolean Algebra. These theorems state that "AND" and "OR" operators are interchangeable through negation.

I) Theorem-1:- The complement of the sum of 2 or more variable is equal to the product of the complement of the variables for two variables 'A' and 'B', the theorem is written in boolean notation as follows:-

$$\overline{A+B} = \bar{A} \cdot \bar{B}$$

A	B	$\bar{A}$	$\bar{B}$	$\bar{A} \cdot \bar{B}$	$A+B$	$\overline{A+B}$
0	0	1	1	1	0	1
0	1	1	0	0	1	0
1	0	0	1	0	1	0
1	1	0	0	0	1	0

II) Theorem-2:- The complement of the product of two or more variables is equal to the sum of the complement of the variables for two variable 'x' and 'y', this theorem is written as in boolean Algebra.

$$\overline{x \cdot y} = \bar{x} + \bar{y}$$

x	y	$x \cdot y$	$\overline{x \cdot y}$	$\bar{x}$	$\bar{y}$	$\bar{x} + \bar{y}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

→ The generalised form of De-Morgan's Theorem:-

$$\overline{x_1 + x_2 + x_3 + \dots + x_n} = \bar{x}_1 \cdot \bar{x}_2 \cdot \bar{x}_3 \cdot \bar{x}_4 \cdot \dots \bar{x}_n$$

$$\overline{x_1 \cdot x_2 \cdot x_3 \cdot \dots \cdot x_n} = \bar{x}_1 + \bar{x}_2 + \bar{x}_3 + \dots + \bar{x}_n$$

→ Boolean funcⁿ :- A Boolean funcⁿ is an expression formed with Binary variables, the 2 Binary Operators "OR" & "AND", the unary operator "NOT", Parenthesis and equal sign.

$$w = x \cdot \bar{y} \cdot z$$

Here, the var. w is a funcⁿ of x , y & z ,

Written as :- $w = F(x, y, z)$.

x	y	z	w
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

→ It is possible to find 2 Algebraic expressions that specify the same boolean funcⁿ.

eg:- Consider these 2 eqⁿs:-

$$1) F_1 = \bar{x} \cdot \bar{y} \cdot z + \bar{x} \cdot y \cdot z + x \cdot \bar{y}$$

$$2) F_2 = x \cdot \bar{y} + \bar{x} \cdot z$$

x	y	z	F ₁	F ₂
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	0	0
1	1	1	0	0

→ Need of Reduction:- When a boolean funcⁿ is implemented with logic gates, each var. in the funcⁿ designates an input to a gate and each term is implemented with a gate, thus for a given Boolean funcⁿ, the minimization of the no. of var. and the no. of terms will result in a less equipments, for minimizing the no. of var. in a given boolean funcⁿ we will use the method of Algebraic manipulation.

$$\text{eg:- a) } x + \bar{x} \cdot \bar{y} \Rightarrow (x + \bar{x}) \cdot (x + \bar{y}) \quad [x + y \cdot z = (x + y) \cdot (x + z)] \\ = 1 \cdot (x + \bar{y}) \quad [x + \bar{x} = 1]$$

$$\text{b) } x(\bar{x} + y) \Rightarrow x \cdot \bar{x} + x \cdot y \quad [x \cdot (y + z) = x \cdot y + x \cdot z] \\ = 0 + x \cdot y \quad [x \cdot \bar{x} = 0] \\ = x \cdot y$$

$$\begin{aligned}
 & \text{d) } \bar{x} \cdot \bar{y} \cdot z + \bar{x} \cdot y \cdot \bar{z} + x \cdot \bar{y} \\
 & \quad \bar{x} \cdot \bar{z} (\bar{y} + y) + x \cdot \bar{y} \quad \left[x \cdot (y + \bar{y}) = x \cdot y + x \cdot \bar{y} \right] \\
 & \quad x \cdot z \cdot 1 + x \cdot \bar{y} \quad \left[x + \bar{x} = 1 \right] \\
 & \quad \bar{x} \cdot \bar{z} + x \cdot \bar{y}
 \end{aligned}$$

→ Complement of a funcⁿ - the comp. of a func "F" is "F̄" and it is obtained by interchanging 0 for 1 and 1 for 0 in the truth table

$$F = x \cdot \bar{y} + \bar{x} \cdot z$$

	x	y	z	F	F̄	̄x	̄y	x · ̄y	̄x · z
	0	0	0	0	1	1	1	0 + 0 = 0	
=	0	0	1	1	0	1	1	0 + 1 = 1	
	0	1	0	0	1	1	0	0 + 0 = 0	
=	0	1	1	1	0	1	0	0 + 1 = 1	
=	1	0	0	1	0	0	1	1 + 0 = 1	
=	1	0	1	1	0	0	1	1 + 0 = 1	
	1	1	0	0	1	0	0	0 + 0 = 0	
	1	1	1	0	1	0	0	0 + 0 = 0	

*Algebraically, the comp. of a funcⁿ may be derived through De-Morgan's Theorem.

$$\begin{aligned}
 F_1 &= \bar{x} \cdot y \cdot \bar{z} + \bar{x} \cdot \bar{y} \cdot z \\
 \bar{F}_1 &= \overline{\bar{x} \cdot y \cdot \bar{z} + \bar{x} \cdot \bar{y} \cdot z} = (\overline{\bar{x} \cdot y \cdot \bar{z}}) \cdot (\overline{\bar{x} \cdot \bar{y} \cdot z})
 \end{aligned}$$

$$= (\overline{\bar{x}} + \bar{y} + \bar{\bar{z}}) \cdot (\overline{\bar{x}} + \bar{\bar{y}} + \bar{z}) = \boxed{(x + \bar{y} + z) \cdot (x + y + \bar{z})}$$

$$F_2 = x \cdot (y \cdot \bar{z} + y \cdot z)$$

$$\bar{F}_2 = \overline{x \cdot (y \cdot \bar{z} + y \cdot z)} = \bar{x} + (\bar{y} + \bar{\bar{z}} \cdot \bar{y} + \bar{z})$$

$$\bar{F}_2 = \boxed{\bar{x} + (\bar{y} + z) \cdot (\bar{y} + \bar{z})}$$

→ Canonical Form 1-

1) Min Term & Max Term.

→ A Binary variable may appear either in normal form (x) or its complement form ($\bar{x}$).

⇒ Now, consider two B. Values x & y combined with an AND operation, since each var. may appear in either,

Here, 1 → Unbarred There are four possible Combos- $x \cdot y, \bar{x} \cdot \bar{y}, \bar{x} \cdot y, x \cdot \bar{y}$.
0 → Barred. (1 1) (0 0) (0 1) (1 0)

each of these four terms are AND terms, Min term or Std. product.

Symbol :- m.

⇒ Similarly, these values combined with an OR operation.

Here, These are the four possible Combos- $x+y, \bar{x}+y, x+\bar{y}, \bar{x}+\bar{y}$
1 → Barred (0 0) (1 0) (0 1) (1 1)
0 → Unbarred each of these four terms are OR terms, Max term or Std. sum.

Variables			Min term		Max term	
x	y	z	Term		Term	
0	0	0	$\bar{x} \cdot \bar{y} \cdot \bar{z}$	m_0	$x + y + z$	M_0
0	0	1	$\bar{x} \cdot \bar{y} \cdot z$	m_1	$x + y + \bar{z}$	M_1
0	1	0	$\bar{x} \cdot y \cdot \bar{z}$	m_2	$x + \bar{y} + z$	M_2
0	1	1	$\bar{x} \cdot y \cdot z$	m_3	$x + \bar{y} + \bar{z}$	M_3
1	0	0	$x \cdot \bar{y} \cdot \bar{z}$	m_4	$\bar{x} + y + z$	M_4
1	0	1	$x \cdot \bar{y} \cdot z$	m_5	$\bar{x} + y + \bar{z}$	M_5
1	1	0	$x \cdot y \cdot \bar{z}$	m_6	$\bar{x} + \bar{y} + z$	M_6
1	1	1	$x \cdot y \cdot z$	m_7	$\bar{x} + \bar{y} + \bar{z}$	M_7

Note- Each Max term is complementary of its corresponding Min term and vice-versa.

→ Sum of products :- A sum of product expression is a product term (min term) or several product terms (min term) logically added (OR) together.

for egr- the expression $x \cdot \bar{y} + \bar{x} \cdot y$ is sum of product expression.

Steps to express a boolean funcⁿ in its sum of product form.

- 1) → Construct a Truth Table for given boolean funcⁿ
- 2) Find all the combination where value of F=1 ↗ if 1 ⇒ F
↘ if 0 ⇒ F
- 3) Combination having F=1 are all the min terms obtained

x	0	0	0	0	1	1	1	1
y	0	0	1	1	0	0	1	1
z	0	1	0	1	0	1	0	1
F	0	1	0	0	1	0	0	1
	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7

eg- $F = \bar{x} \cdot \bar{y} \cdot z + x \cdot \bar{y} \cdot \bar{z} + x \cdot y \cdot z$
 $(0\ 0\ 1)\ (1\ 0\ 0)\ (1\ 1\ 1)$

$F = m_1 + m_4 + m_7$

$F(x, y, z) = \Sigma(1, 4, 7)$

→ Product of Sum: A Product of Sum expression is a (AND) (OR) Sumterm (Maxterm) or several Sumterm logically multiplied together

for egr- $(\bar{x} + y) \cdot (x + \bar{y})$ is Product of Sum

Steps to express a boolean funcⁿ in its product of sum form

- 1) Construct a truth table.
- 2) find all the combination where value of $F=0$ $\begin{matrix} 0 \rightarrow F \\ 1 \rightarrow F \end{matrix}$
- 3) Comb. having $F=0$ are all the Maxterms obtained.

x	y	z	F	
✓ 0	0	0	0	eg- $F_1 = (x+y+z) \cdot (x+\bar{y}+z) \cdot (x+\bar{y}+\bar{z}) \cdot (\bar{x}+y+\bar{z}) \cdot (\bar{x}+\bar{y}+z)$
0	0	1	1	
✓ 0	1	0	0	$F_1 = M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6$
✓ 0	1	1	0	
1	0	0	1	$F_1(x,y,z) = \pi(0,2,3,5,6)$
✓ 1	0	1	0	
✓ 1	1	0	0	
1	1	1	1	

→ Basic Building Blocks of Digital Circuits.

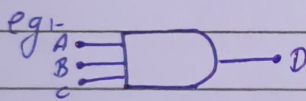
Logic Gates:- They are electronic circuits that perform operations on one or more input signals to produce a single output signal.

I) AND Gates:- Physical realisation of logical mul. operation.
It is an electronic circuit that generates output signal 1 only-if all Input signals are 1.



$$C = A \cdot B$$

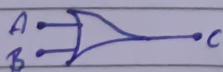
A	B	C
0	0	0
0	1	0
1	0	0
1	1	1 ✓



$$D = A \cdot B \cdot C$$

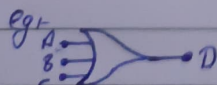
A	0	0	0	0	1	1	1	1
B	0	0	1	1	0	0	1	1
C	0	1	0	1	0	1	0	1
D	0	0	0	0	0	0	0	1

II) OR Gate:- P. Realisation of logical addition operation.
It is an e-circuit that generates output signal 1 even-if-only one of Input signal is 1.



$$C = A + B$$

A	B	C
0	0	0
0	1	1 ✓
1	0	1 ✓
1	1	1 ✓



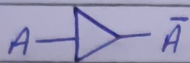
$$D = A + B + C$$

A	0	0	0	0	1	1	1	1
B	0	0	1	1	0	0	1	1
C	0	1	0	1	0	1	0	1
D	0	1	1	1	1	1	1	1



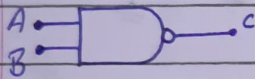
III) NOT Gate - P.R of complement operation. e-circuit that generates output signal opposite/reverse of Input signal.

→ Also called inverter as it inverts the output.

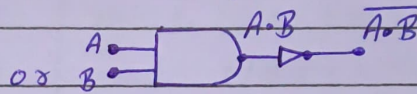


A	$\bar{A}$
0	1
1	0

IV) NAND Gate - Complement of "AND" gate. It shows inverse value of AND gate.



$$C = \overline{A \cdot B}$$

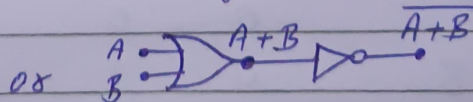


A	B	$A \cdot B$	$\overline{A \cdot B}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

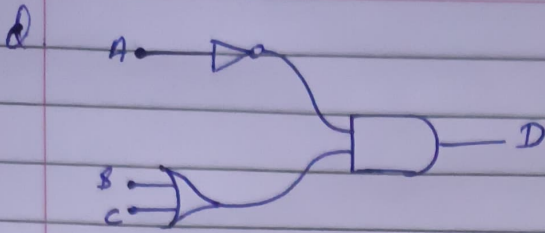
V) NOR Gate - Complement of "OR" gate. It shows inverse value of OR gate.



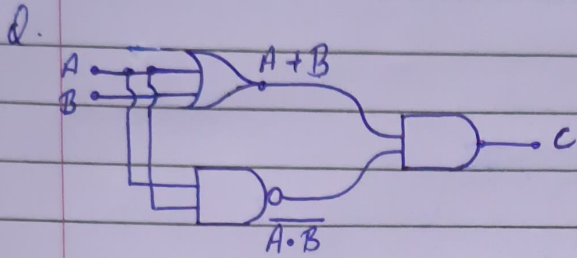
$$C = \overline{A + B}$$



A	B	$A + B$	$\overline{A + B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0



Ans $D = \bar{A} \cdot B + C$



Ans $C = (A + B) \cdot (\overline{A \cdot B})$